

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30724
	Apellidos y nombres de los estudiantes: Díaz Stiven Leiton Jose Manosalvas Gabriel

ANTECEDENTES

1. La administración del Condominio *La Primavera* presenta limitaciones operativas debido al uso de herramientas manuales, principalmente hojas de cálculo, para la gestión de copropietarios, expensas y pagos. Esta situación ha generado problemas como retrasos en la actualización de información, inconsistencias en los registros y una limitada transparencia en el control financiero hacia los residentes.
2. Asimismo, procesos críticos como el cálculo de la mora del 12% se realizan de forma manual, incrementando el riesgo de errores y dificultando la aplicación uniforme de las reglas administrativas. A esto se suma la ausencia de un canal automatizado de comunicación, lo que limita la interacción oportuna entre la administración y los copropietarios.
3. En este contexto, se propone el desarrollo de **AliGest**, una plataforma tecnológica orientada a la gestión de condominios, que permitirá centralizar la información, automatizar procesos clave y mejorar la comunicación entre los diferentes roles del sistema, principalmente el administrador y los copropietarios.
4. En la Guía 1 se definieron los requisitos funcionales y los casos de uso de nivel 0, estableciendo qué debe hacer el sistema. En la presente Guía 2, el enfoque se orienta al análisis estructural y dinámico del sistema, definiendo cómo se implementarán dichas funcionalidades mediante clases de análisis (Boundary, Control y Entity) y diagramas de secuencia de nivel 0.

OBJETIVO

1. Modelar la estructura y el comportamiento del sistema **AliGest** mediante el uso de diagramas UML desarrollados en **PlantUML**, representando las clases de análisis (Boundary, Control y Entity) y las interacciones entre estas a través de diagramas de secuencia de nivel 0.

Adicionalmente, garantizar la trazabilidad entre los requisitos funcionales, los actores del sistema (administrador y copropietario), los casos de uso y los modelos

generados, mediante una matriz de relación que evidencie la coherencia, consistencia y alineación del sistema con las necesidades identificadas.

DESARROLLO

GUÍA 2 - MÓDULO PAGO.

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

1. Requisitos Funcionales

- RF-01 El sistema debe gestionar el acceso, autenticación mediante credenciales, recuperación de contraseña y configuración de roles.
- RF-02 El sistema debe permitir administrar la información de los copropietarios (incluyendo importación masiva) y generar reportes basados en roles.
- RF-03 El sistema debe gestionar el proceso completo de registro, validación y control de pagos, incluyendo el cálculo automático del recargo del 12% por mora.
- RF-04 El sistema debe enviar automáticamente mensajes de WhatsApp a los copropietarios para notificar aprobación, rechazo o sanción por mora.

2. Diagrama de Casos de Uso

@startuml

left to right direction

skinparam packageStyle rectangle

skinparam shadowing false

actor "Administrador" as Admin

actor "Copropietario" as Copro

actor "API WhatsApp" as API_WA <<Sistema Externo>>

rectangle "Módulos Principales" {

 usecase "RF-01: Administrar Sistema" as UC1

 usecase "RF-02: Gestionar Copropietarios" as UC2

 usecase "RF-03: Implementar Pagos" as UC3

 usecase "RF-04: Enviar Notificaciones" as UC4

}

Admin --> UC1

Admin --> UC2

Admin --> UC3

Copro --> UC3

UC3 ..> UC4 : <<include>>

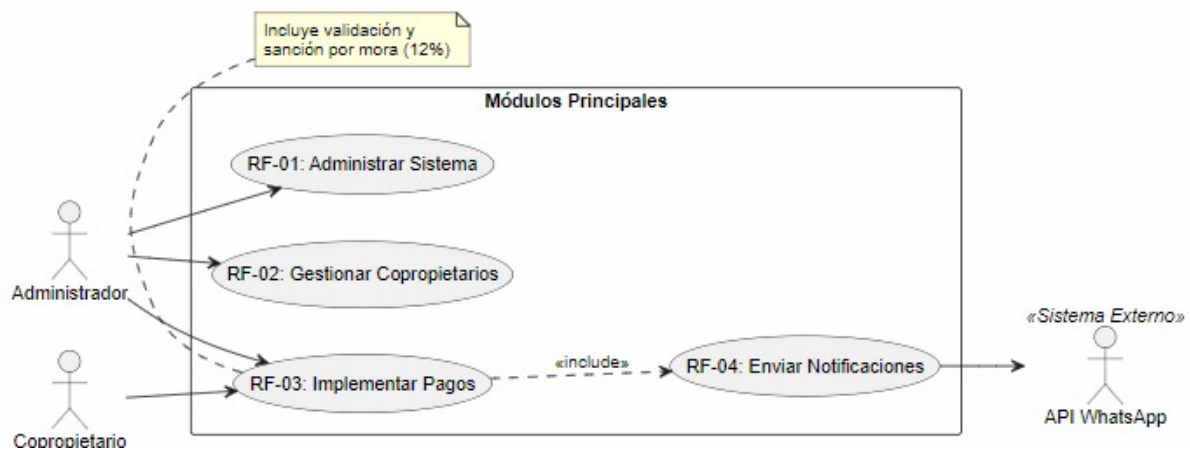
UC4 --> API_WA

```

note bottom of UC3
  Incluye validación y
  sanción por mora (12%)
end note
@enduml

```

Resultado



CLASES DE ANÁLISIS

A continuación, el diagrama de clases modelando la Frontera, Control, Entidad y Repositorio para la gestión de pagos.

```

@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

'=====
' VISTAS (BOUNDARY)
'=====

class "PantallaAdministrarSistema" <<boundary>> {
- txtConfiguracion: String
- txtEstado: String
+ clickConfigurar()
+ clickActualizar()
+ mostrarMensaje(mensaje: String)
+ mostrarEstadoSistema()
}

class "PantallaCrudCopropietario" <<boundary>> {
- txtCedula: String
- txtNombre: String
- txtDepartamento: String
- txtEstadoPago: String
+ clickAgregar()
}

```

```

+ clickActualizar()
+ clickEliminar()
+ clickMostrarTodos()
+ mostrarMensaje(mensaje: String)
+ mostrarTabla(copropietarios: List<Copropietario>)
}

```

```

class "PantallaGestionPagos" <<boundary>> {
- txtIdPago: String
- txtMonto: double
- txtFecha: Date
+ clickRegistrarPago()
+ clickConsultarPago()
+ mostrarConfirmacion()
+ mostrarEstadoPago()
}

```

```

'=====
' CONTROLADORES
'=====

```

```

class "ControlAdministracion" <<control>> {
+ configurarSistema(configuracion: String): Resultado
+ actualizarSistema(): Resultado
+ validarConfiguracion(): boolean
}

```

```

class "ControlCopropietario" <<control>> {
+ agregarCopropietario(cedula: String, nombre: String, departamento: String): Resultado
+ actualizarCopropietario(cedula: String, nombre: String, departamento: String): Resultado
+ eliminarCopropietario(cedula: String): Resultado
+ listarCopropietarios(): List<Copropietario>
+ validarDatos(): boolean
}

```

```

class "ControlPago" <<control>> {
+ registrarPago(idPago: String, monto: double): Resultado
+ calcularMora(monto: double): double
+ validarPago(): boolean
+ enviarNotificacion(): void
}

```

```

'=====
' ENTIDADES (MODELO)
'=====

```

```

class "Sistema" <<entity>> {
- estado: String
- configuracion: String
+ actualizarEstado()
}

```

```
+ guardarConfiguracion()
}
```

```
class "Copropietario" <<entity>> {
  - cedula: String
  - nombre: String
  - departamento: String
  - estadoPago: String
  + getCedula(): String
  + getNombre(): String
  + getDepartamento(): String
}
```

```
class "Pago" <<entity>> {
  - idPago: String
  - monto: double
  - mora: double
  - fecha: Date
  + calcularMora(): double
  + getMonto(): double
}
```

```
class "NotificacionWhatsApp" <<entity>> {
  - numeroDestino: String
  - mensaje: String
  + enviarMensaje(): boolean
}
```

```
'=====
' REPOSITORIOS
'=====
```

```
class "RepositorioCopropietario" <<repository>> {
  + guardar(c: Copropietario): void
  + buscarPorCedula(cedula: String): Copropietario
  + actualizar(c: Copropietario): void
  + eliminar(cedula: String): void
  + listarTodos(): List<Copropietario>
}
```

```
class "RepositorioPago" <<repository>> {
  + guardarPago(p: Pago): void
  + buscarPago(idPago: String): Pago
  + listarPagos(): List<Pago>
}
```

```
'=====
' RELACIONES MVC
'=====
```

PantallaAdministrarSistema --> ControlAdministracion : solicita operación
ControlAdministracion --> Sistema : administra configuración

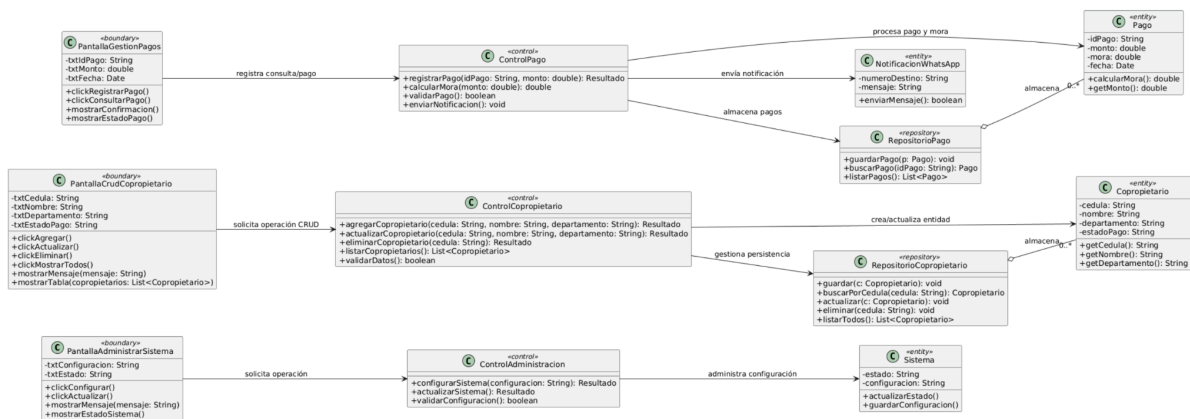
PantallaCrudCopropietario --> ControlCopropietario : solicita operación CRUD
ControlCopropietario --> Copropietario : crea/actualiza entidad
ControlCopropietario --> RepositorioCopropietario : gestiona persistencia

PantallaGestionPagos --> ControlPago : registra consulta/pago
ControlPago --> Pago : procesa pago y mora
ControlPago --> RepositorioPago : almacena pagos
ControlPago --> NotificacionWhatsApp : envía notificación

RepositorioCopropietario o-- "0..*" Copropietario : almacena
RepositorioPago o-- "0..*" Pago : almacena

@enduml

Resultado



SECUENCIAS DE ANÁLISIS

Secuencia 1: Administrar Sistema (RF-01 Iniciar Sesión)

@startuml

title Secuencia de análisis - Administrar sistema

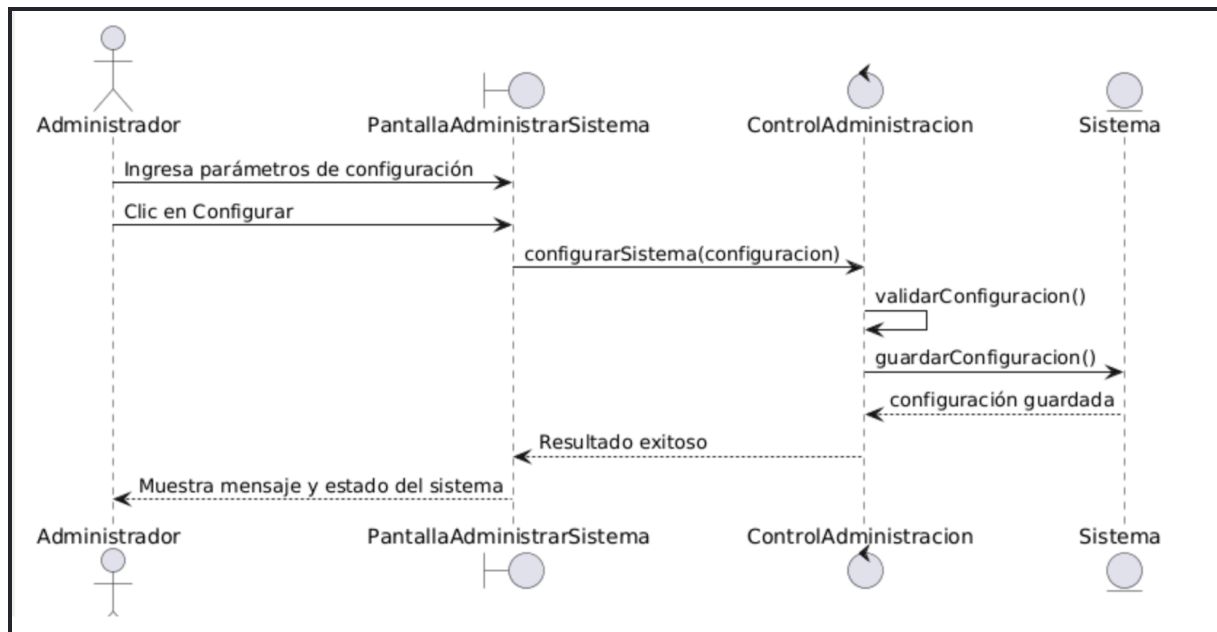
actor "Administrador" as Admin
boundary "PantallaAdministrarSistema" as UI
control "ControlAdministracion" as Ctrl
entity "Sistema" as Sis

Admin -> UI : Ingresa parámetros de configuración
Admin -> UI : Clic en Configurar
UI -> Ctrl : configurarSistema(configuracion)
Ctrl -> Ctrl : validarConfiguracion()
Ctrl -> Sis : guardarConfiguracion()

Sis --> Ctrl : configuración guardada
 Ctrl --> UI : Resultado exitoso
 UI --> Admin : Muestra mensaje y estado del sistema

@enduml

Resultado



Secuencia 2: Gestionar Copropietarios (RF-02 Importar/Agregar)

@startuml

title Secuencia de análisis - Gestionar copropietarios

actor "Administrador" as Admin
 boundary "PantallaCrudCoproietario" as UI
 control "ControlCoproietario" as Ctrl
 entity "Copropietario" as Copro
 database "RepositorioCoproietario" as Repo

Admin -> UI : Ingresa cédula, nombre y departamento
 Admin -> UI : Clic en Agregar
 UI -> Ctrl : agregarCoproietario(cedula,nombre,departamento)

Ctrl -> Ctrl : validarDatos()

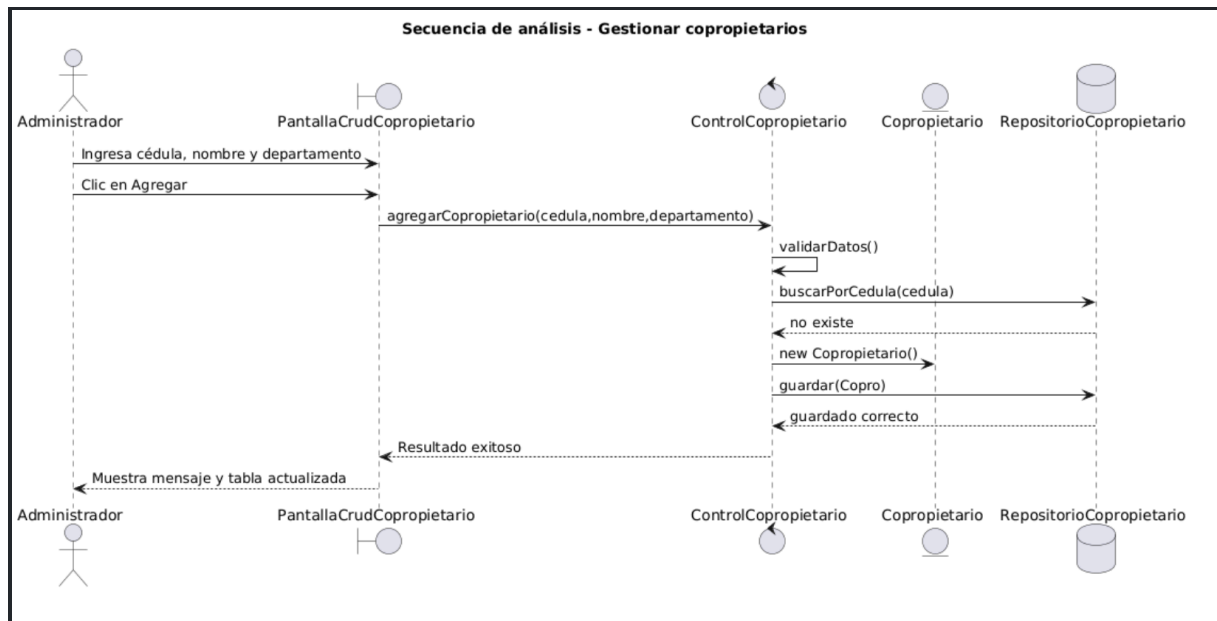
Ctrl -> Repo : buscarPorCedula(cedula)
 Repo --> Ctrl : no existe

Ctrl -> Copro : new Copropietario()
 Ctrl -> Repo : guardar(Copro)

Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada

@enduml

Resultado



Secuencia 3: Implementar Pagos (RF-03 Validar y Aplicar Mora)

@startuml

title Secuencia de análisis - Implementar pagos

actor "Copropietario" as User

boundary "PantallaGestionPagos" as UI

control "ControlPago" as Ctrl

entity "Pago" as Pago

database "RepositorioPago" as Repo

User -> UI : Ingresa datos del pago

User -> UI : Clic en Registrar Pago

UI -> Ctrl : registrarPago(idPago, monto)

Ctrl -> Ctrl : validarPago()

Ctrl -> Pago : calcularMora()

Ctrl -> Repo : guardarPago(Pago)

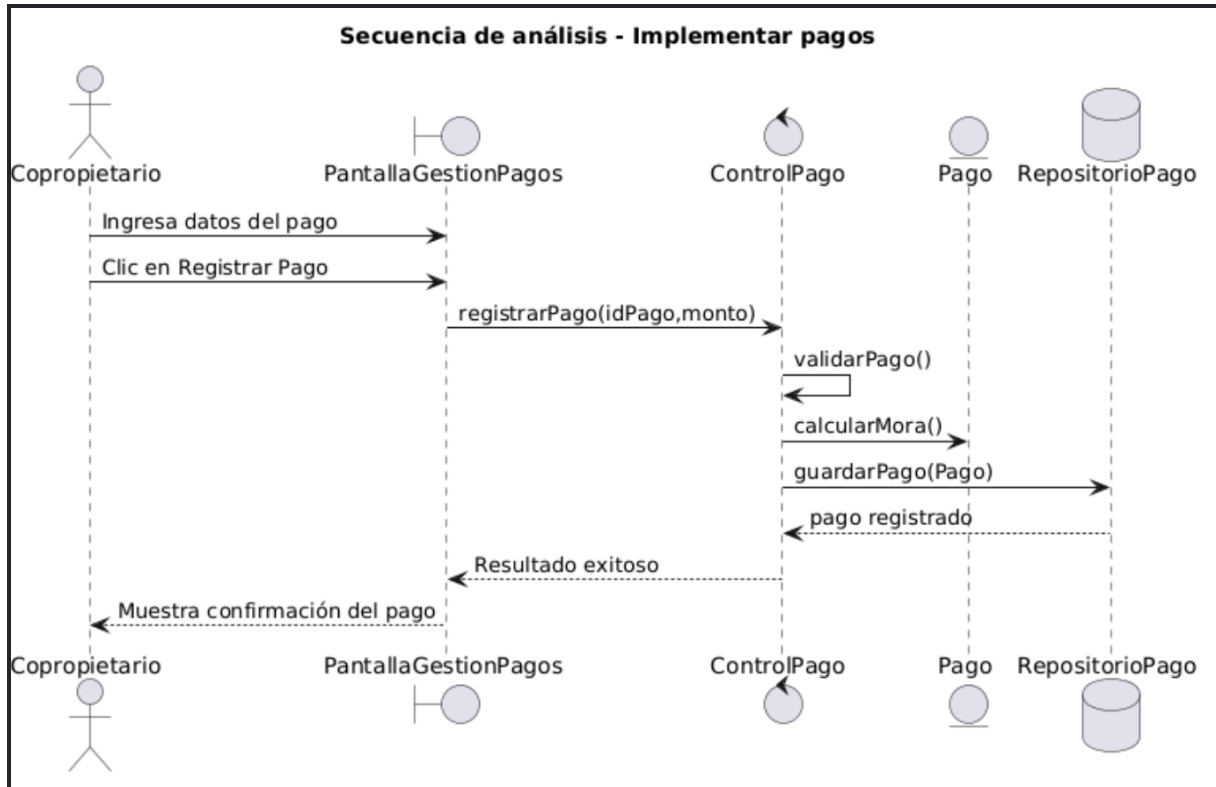
Repo --> Ctrl : pago registrado

Ctrl --> UI : Resultado exitoso

UI --> User : Muestra confirmación del pago

@enduml

Resultado



Secuencia 4: Enviar Notificaciones (RF-04 Vía WhatsApp)

@startuml

title Secuencia de análisis - Enviar notificaciones

actor "Administrador" as Admin

boundary "PantallaGestionPagos" as UI

control "ControlPago" as Ctrl

entity "NotificacionWhatsApp" as Noti

collections "API WhatsApp" as API

Admin -> UI : Solicita envío de notificación

UI -> Ctrl : enviarNotificacion()

Ctrl -> Noti : generarMensaje()

Ctrl -> API : enviarMensaje(numero, mensaje)

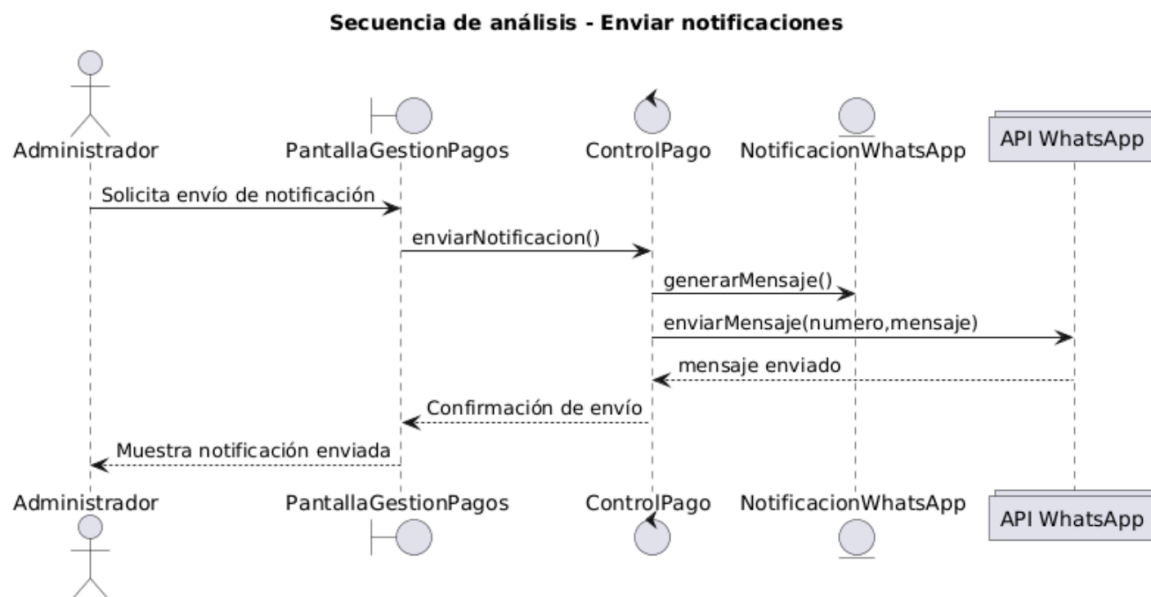
API --> Ctrl : mensaje enviado

Ctrl --> UI : Confirmación de envío

UI --> Admin : Muestra notificación enviada

@enduml

Resultado



3. MATRIZ DE TRAZABILIDAD

RF	Caso de Uso	Boundary (Frontera)	Control	Entity / Repositorio	Reglas de Negocio	Diagrama de Secuencia
RF-01	Administrar Sistema (Autenticación)	LoginView: Captura de usuario y contraseña.	AuthController: Gestiona autenticación, sesiones y permisos.	Usuario / UsuarioRepository: Consulta credenciales y estado del usuario.	- Bloqueo tras 3 intentos fallidos. - Encriptación de contraseñas. - Manejo de sesiones activas.	Usuario ingresa credenciales → Validación de campos → Envío a AuthController → Consulta a BD → Verificación de contraseña → Generación de sesión → Respuesta de acceso (éxito/error).
RF-02	Gestionar Copropietarios	CopropietarioView: Registro, edición, eliminación, consulta e importación masiva.	CopropietarioController: Valida datos y gestiona operaciones CRUD.	Copropietario / CopropietarioRepository: Almacena y actualiza datos de residentes.	- Validación de cédula y formato de email. - No duplicidad de registros. - Integridad de datos obligatorios.	Usuario ingresa o importa datos → Validación de formato → Envío a controlador → Verificación de duplicados → Guardado/actualización en BD → Confirmación de operación.
RF-03	Implementar Pagos	PagoView: Registro de pago, ingreso de monto y carga de comprobante.	PagoController: Procesa pagos, calcula mora y actualiza estado financiero.	Pago, Factura / PagoRepository: Guarda pagos y actualiza deudas.	- Aplicar recargo del 12% por mora. - Validar monto mínimo. - Asociar pago a factura pendiente.	Usuario registra pago → Validación de datos → Consulta de deuda → Cálculo de mora → Generación de total → Registro de pago → Actualización de estado → Confirmación.
RF-04	Enviar Notificaciones	NotificacionView / WhatsAppAPI: Interfaz y servicio externo de mensajería.	NotificacionController: Genera, formatea y envía mensajes.	Notificacion (opcional) / Consulta de datos en repositorio.	Validación de número telefónico. - Formato estándar de mensaje. - Envío solo ante eventos relevantes.	Evento del sistema → Generación de mensaje → Consulta de datos → Formateo → Envío a API externa → Recepción de respuesta → Confirmación de envío.

CONCLUSIÓN TÉCNICA

El modelo desarrollado evidencia una adecuada trazabilidad entre los requisitos funcionales, los casos de uso y los diagramas de análisis, permitiendo validar la coherencia del sistema propuesto. La separación en clases de tipo Boundary, Control y Entity facilita la comprensión de las responsabilidades dentro del sistema, mientras que los diagramas de secuencia permiten visualizar el flujo de interacción entre los componentes.

Adicionalmente, la integración de procesos como la validación de pagos y el envío de notificaciones externas demuestra la capacidad del modelo para representar escenarios reales de operación. En conjunto, el uso de PlantUML permitió estructurar de forma clara y consistente tanto la arquitectura como el comportamiento del sistema AliGest.

ANEXOS (RÚBRICA DE LA GUÍA)

Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad RF-CU	Relación clara entre requisitos funcionales, actor, caso de uso y criterio de aceptación.	5 pts	Matriz RF-CU completa.
Modelado de casos de uso	Representación correcta de actor, límite del sistema, casos de uso y relaciones.	5 pts	Diagrama PlantUML generado.
Claridad del análisis	Selección pertinente de funcionalidades de nivel 0 y nombres orientados a objetivos del usuario.	4 pts	Casos de uso redactados claramente.
Uso correcto de PlantUML	Código funcional, ordenado y comprensible.	3 pts	Código sin errores de sintaxis.

Presentación técnica	Documento ordenado, con evidencias, explicación breve y coherencia visual.	3 pts	Entrega final documentada.
----------------------	--	-------	----------------------------

Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Requisitos funcionales	Listado de RF de nivel 0 para el CRUD de Estudiante.	Tabla RF.
Diagrama de casos de uso	Código PlantUML y captura del diagrama generado.	Código + imagen.
Matriz de trazabilidad	Relación RF, actor, caso de uso, prioridad y criterio de aceptación.	Tabla RF-CU.
Conclusión técnica	Reflexión breve sobre trazabilidad, consistencia y utilidad del modelo.	Texto de cierre.